



HCMUS

Viet Nam National University
Ho Chi Minh City
University of Science

fit@hcmus

Seminar: Design Patterns

Topic: Visitor pattern

Course: CS202- Programming Systems

Supervisor:

PhD. DINH Ba Tien

MSc. TRUONG Phuoc Loc

MSc. HO Tuan Thanh

Group 03

Full Name	Student ID
Đỗ Viết Hoàng Long	25125024
Trần Như Khải	25125045
Trần Đăng Khoa	25125056
Trần Minh Khoa	25125057

Contents

1	Introduction	2
2	Problem Statement	3
2.1	The Context and Naive Solution	3
2.2	Shortcomings of the Naive Solution	4
3	Proposed Solution: Classic Visitor Pattern	5
3.1	Structural Overview	5
3.2	Conceptual Implementation	5
3.3	The Core Mechanism: Double Dispatch	5
4	Implementation: Classic Visitor in C++	6
4.1	Core Interfaces	6
4.2	Concrete Implementations	7
4.3	Client Code Execution	8
5	Modern Visitor Variants	9
5.1	Modern C++ Approach: <code>std::variant</code> and <code>std::visit</code>	9
5.1.1	Overview of <code>std::variant</code>	9
5.1.2	Mechanism: Functors and Runtime Dispatching	9
5.2	The Acyclic Visitor Pattern	10
5.2.1	Structural Overview	10
5.2.2	Mechanism: Cross-Casting	10
6	Evaluation	12
6.1	Experimental Environment	12
6.2	Measurement Methodology	12
6.3	Disclaimer	12
6.4	Benchmark Results	13
7	Discussion	13
7.1	Classic Visitor Pattern	13
7.2	Modern C++ Approach (<code>std::variant</code> and <code>std::visit</code>)	14
7.3	The Acyclic Visitor Pattern	14
8	Real-World Applications	15
8.1	Compiler Construction and AST Processing	15
8.2	GUI Hierarchies and Theming Engines	15
8.3	Document Serialization and Exporting	15
9	Quiz	16
10	Conclusion	18
11	References	19
A	Declaration of AI Assistance	20

1 Introduction

C++ is a highly versatile programming language that provides profound support for Object-Oriented Programming (OOP) paradigms, including encapsulation, inheritance, and polymorphism. As software systems expand, developers frequently construct complex object structures, such as intricate data trees or user interface component graphs. Managing operations across these heterogeneous collections often introduces significant architectural challenges.

To address recurring design problems within OOP architectures, *Design Patterns* serve as proven, standardized templates representing best practices. Among the three primary classifications of design patterns, **Behavioral Patterns** focus specifically on the effective assignment of responsibilities between objects and the intricate communication mechanisms that govern their interactions, ensuring loose coupling and high cohesion.

The **Visitor Pattern** is a prominent behavioral design pattern whose primary intent is to separate algorithms or operations from the object structures on which they operate. In traditional object-oriented design, introducing a new behavior across a complex hierarchy of elements often leads to rigid and hard-to-maintain codebases. The Visitor pattern offers an elegant architectural approach to overcome this limitation, allowing developers to extend the functionality of a system dynamically.

The objective of this report is to present a study of the Visitor design pattern within the context of C++ development. It systematically explores the fundamental problem the pattern addresses, its structural architecture, the underlying mechanics of double dispatch, modern C++ implementation variants, and its pragmatic applications in contemporary software engineering.

2 Problem Statement

2.1 The Context and Naive Solution

In real-world software development, object hierarchies are rarely static; they frequently evolve as new operational requirements emerge. To illustrate this, consider a typical graphics application managing a hierarchy of geometric elements, such as **Circle** and **Rectangle**, which both derive from a common **Shape** interface.

Suppose a new business requirement dictates that the system must now serialize these elements to an external file. Under a conventional Object-Oriented paradigm—often deemed the naive solution—the intuitive approach is to embed a new pure virtual method, such as `save2File(std::string& file)`, directly into the base **Shape** class. Consequently, this method must be overridden across all concrete derived classes.

While this approach ostensibly leverages C++’s dynamic polymorphism, allowing the new operation direct access to the object’s internal state, it tightly couples orthogonal behaviors (file I/O operations) with the core domain logic (geometric properties).

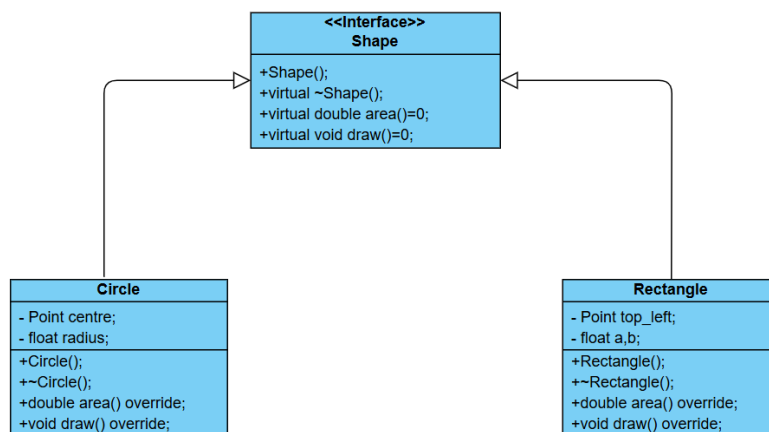


Figure 2.1: Class Diagram illustrating the naive approach: embedding operations directly into the element hierarchy.

The following C++ code snippet demonstrates this rigid structure, where adding a new feature requires modifying the base interface:

```

1 class Shape {
2 public:
3     virtual ~Shape() = default;
4     // Original Shape logic
5     virtual void draw() = 0;
6     virtual double area()=0;
7
8     // NAIVE APPROACH: Adding a new operation directly modifies the base
9     // interface
10    virtual void save2File(string& file) = 0;
11 };
12
13 class Circle : public Shape {
14 private:
15     Point centre;
16     float radius;
  
```

```
16 public:
17     void draw() override { /* ... */ }
18     double area() override { /* ... */ }
19
20     // new operator override
21     void save2File(string& file) override { /* ... */ }
22 };
23
24 class Rectangle : public Shape {
25 private:
26     Point top_left;
27     float a,b;
28 public:
29     void draw() override { /* ... */ }
30     double area() override { /* ... */ }
31
32     // new operator override
33     void save2File(string& file) override { /* ... */ }
34 };
```

Listing 1: Naive solution violating design principles.

2.2 Shortcomings of the Naive Solution

While the naive approach appears straightforward initially, it introduces critical architectural and operational bottlenecks, particularly within C++ environments:

- **Violation of the Single Responsibility Principle (SRP):** Forcing business logic classes to implement unrelated operations bloats the entities with orthogonal concerns. For instance, a shape class should not be burdened with the responsibility of saving data to external file.
- **Violation of the Open/Closed Principle (OCP):** Every time a new operation is introduced, the core base class and all its derived classes must be modified. The system is fundamentally “open for modification,” making the codebase brittle and prone to regression bugs.
- **Recompilation Cascade and ABI Breakage:** This is a severe operational issue in C++. Modifying a base class header by adding new virtual functions alters the Virtual Method Table (`vtable`) layout, thereby breaking Application Binary Interface (ABI) compatibility. Consequently, every translation unit that includes the modified header must be recompiled, drastically increasing build times in large-scale projects.
- **Restricted Source Code Accessibility:** In many practical scenarios, developers integrate third-party libraries, legacy codebases, or compiled SDKs. In such instances, modifying the original class structures is strictly prohibited or physically impossible due to a lack of source code access or organizational permissions.

3 Proposed Solution: Classic Visitor Pattern

3.1 Structural Overview

The Visitor design pattern is strategically proposed to resolve the architectural limitations highlighted in the previous section. It achieves this by extracting the newly required operations from the base class hierarchy and encapsulating them within a distinct “Visitor” object. This decoupling ensures that the primary element classes remain relatively stable and strictly adhere to the Open/Closed Principle.

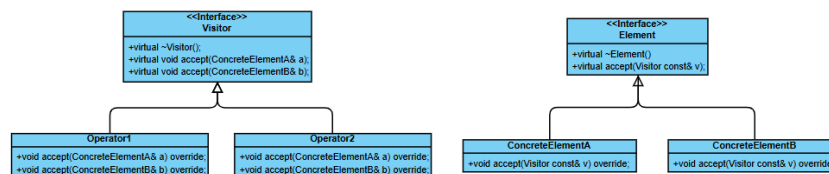


Figure 3.1: UML Class Diagram of the Classic Visitor Pattern.

3.2 Conceptual Implementation

To illustrate the operational flow and the underlying dispatch mechanism, consider the following minimal conceptual implementation. This snippet strips away the business logic to focus entirely on the structural relationship between the `Visitor` interface and the `Shape` hierarchy:

```

1 class Visitor {
2 public:
3     virtual void visit(Circle&) = 0;
4     virtual void visit(Rectangle&) = 0;
5 };
6
7 class Shape {
8 public:
9     virtual void accept(Visitor&) = 0;
10 };
11
12 class Circle : public Shape {
13 public:
14     void accept(Visitor& visitor) override {
15         // The element passes itself to the visitor
16         visitor.visit(*this);
17     }
18 };
  
```

Listing 2: Conceptual implementation illustrating the Visitor structure.

3.3 The Core Mechanism: Double Dispatch

At its core, the Visitor pattern leverages the concept of *Double Dispatch* facilitated by dynamic polymorphism and method overloading. Inherently, C++ only supports *Single Dispatch*, meaning that polymorphic method resolution is based solely on the runtime type of the calling object.

The Visitor pattern elegantly circumvents this language limitation by simulating double dispatch through a sequence of two consecutive single dispatch calls, as demonstrated in the conceptual snippet above:

1. **The First Dispatch (Acceptance):** The client invokes the `accept(Visitor&)` method on a polymorphic base pointer (e.g., `Shape*`). Through standard dynamic binding (vtable lookup), the runtime resolves this call to the specific overridden method of the concrete derived class (e.g., `Circle::accept`).
2. **The Second Dispatch (Visitation):** Inside the resolved `accept` method, the concrete element invokes the `visit` method on the passed `Visitor` object, supplying itself (`*this`) as the argument. At this stage, the compiler uses *overload resolution* (at compile time) to statically bind the call to the appropriate `visit` signature (e.g., `visit(Circle&)`). Then, *virtual dispatch* (at runtime) directs the execution to the corresponding method in the concrete visitor. Thus, the Visitor pattern simulates double dispatch through the combination of compile-time overload resolution and runtime virtual dispatch.

4 Implementation: Classic Visitor in C++

To demonstrate the practical application of the theoretical concepts discussed, this section presents a complete, runnable C++ implementation of the Classic Visitor pattern applied to the geometric shapes scenario.

In a professional C++ software architecture, these components are typically distributed across multiple header and source files to minimize compilation dependencies. A critical technical nuance when implementing the Visitor pattern in C++ is the inherently circular dependency between the `Visitor` interface and the `Element` interface. To resolve this, forward declarations must be utilized appropriately. The implementation is logically divided into three primary components: core interfaces, concrete elements and operations, and client execution.

4.1 Core Interfaces

The foundational step involves defining the abstract base classes. Forward declarations of the concrete elements are required before defining the `Visitor` interface, allowing the compiler to recognize the types without requiring their full definitions.

```
1 #pragma once
2 #include <string>
3
4 // Forward declarations to resolve circular dependencies
5 class Circle;
6 class Rectangle;
7
8 // ===== VISITOR HIERARCHY =====
9 class Visitor {
10 public:
11     virtual ~Visitor() = default;
12
13     // Overloaded visit methods for each Concrete Element
14     virtual void visit(Circle& c) = 0;
```

```
15     virtual void visit(Rectangle& r) = 0;
16 };
17
18 // ===== ELEMENT HIERARCHY =====
19 class Shape {
20 public:
21     virtual ~Shape() = default;
22     virtual void draw() = 0;
23     virtual double area() = 0;
24
25     // The core entry point for the Visitor
26     virtual void accept(Visitor& v) = 0;
27 };
```

Listing 3: Core Interfaces (typically Visitor.h and Shape.h)

4.2 Concrete Implementations

Once the abstract contracts are established, the concrete classes can be implemented. The concrete elements (`Circle`, `Rectangle`) provide their domain-specific logic and implement the `accept` method to initiate the second dispatch. Concurrently, the `SaveVisitor` encapsulates the state (e.g., the file path) and the specific serialization logic.

```
1 #include <iostream>
2 // In a real project, this file would include Shape.h and Visitor.h
3
4 // --- Concrete Elements ---
5 class Circle : public Shape {
6 private:
7     float radius;
8 public:
9     explicit Circle(float r) : radius(r) {}
10
11     void draw() override { std::cout << "Drawing Circle\n"; }
12     double area() override { return 3.14159 * radius * radius; }
13
14     // First Dispatch: Resolves to Circle::accept
15     void accept(Visitor& v) override {
16         // Second Dispatch: Calls visit(Circle&)
17         v.visit(*this);
18     }
19 };
20
21 class Rectangle : public Shape {
22 private:
23     float a, b;
24 public:
25     Rectangle(float width, float height) : a(width), b(height) {}
26
27     void draw() override { std::cout << "Drawing Rectangle\n"; }
28     double area() override { return a * b; }
29
30     // First Dispatch: Resolves to Rectangle::accept
31     void accept(Visitor& v) override {
32         // Second Dispatch: Calls visit(Rectangle&)
33         v.visit(*this);
34     }
35 };
```

```
35 };
36
37 // --- Concrete Visitor ---
38 class SaveVisitor : public Visitor {
39 private:
40     std::string filePath;
41 public:
42     // Explicit constructor to initialize the visitor's state
43     explicit SaveVisitor(const std::string& path) : filePath(path) {}
44
45     void visit(Circle& c) override {
46         // Logic for saving a Circle to file
47         std::cout << "Saving Circle data to " << filePath << "...\\n";
48     }
49
50     void visit(Rectangle& r) override {
51         // Logic for saving a Rectangle to file
52         std::cout << "Saving Rectangle data to " << filePath << "...\\n";
53     }
54 };
```

Listing 4: Concrete Classes (Elements and Visitors)

4.3 Client Code Execution

The client code remains entirely decoupled from the concrete visitor logic. It simply iterates through a heterogeneous collection of shapes and applies the desired operation via the `accept` method, showcasing the elegance and flexibility of the architecture.

```
1 #include <vector>
2 #include <memory>
3
4 void practiceUsage() {
5     // Constructing a heterogeneous collection of shapes using smart
    // pointers for RAII
6     std::vector<std::unique_ptr<Shape>> shapes;
7     shapes.push_back(std::make_unique<Circle>(5.0f));
8     shapes.push_back(std::make_unique<Rectangle>(4.0f, 6.0f));
9
10    // Instantiating the new operation with necessary state
11    SaveVisitor saveOp("export_data.xml");
12
13    // Applying the operation across the entire object structure
14    for (const auto& shape : shapes) {
15        // Executes double dispatch
16        shape->accept(saveOp);
17    }
18
19    // No explicit memory cleanup required (handled by std::unique_ptr)
20 }
```

Listing 5: Client Code demonstrating polymorphic iteration and Double Dispatch (main.cpp)

5 Modern Visitor Variants

This section explores two structural alternatives to the Classic Visitor pattern: the modern C++17 `std::variant` approach and the Acyclic Visitor pattern. The focus here is strictly on their architectural layout and operational mechanisms. A detailed comparative analysis of their advantages and trade-offs will be provided in the Discussion section.

5.1 Modern C++ Approach: `std::variant` and `std::visit`

5.1.1 Overview of `std::variant`

Introduced in the C++17 standard, `std::variant` is a type-safe union. It allows a single variable to store exactly one object from a predetermined, finite set of types. Unlike polymorphic pointers that refer to a heap-allocated base class, a `std::variant` stores its active value directly by value and maintains an internal index to track which type is currently active.

5.1.2 Mechanism: Functors and Runtime Dispatching

The implementation utilizing `std::variant` completely changes the dispatch mechanism by removing the abstract base class (`Shape`) and the `accept` methods.

In this architecture, the “Visitor” is constructed as a Functor—a structure containing overloaded `operator()` methods for each concrete type. At runtime, the `std::visit` standard library function acts as the central dispatcher. It performs dispatch based on the active alternative stored in the `std::variant`. In common implementations, this may be realized using a dispatch table or an equivalent optimized mechanism, which automatically extracts the underlying concrete object and invokes the matching `operator()` overload within the functor.

Figure 5.1: Structural representation of the `std::variant` approach, operating without base classes.

```
1 #include <iostream>
2 #include <variant>
3 #include <vector>
4 #include <string>
5
6 // Independent element classes (No Base Shape required)
7 class Circle {
8 public:
9     float radius;
10    explicit Circle(float r) : radius(r) {}
11 };
12
13 class Rectangle {
14 public:
15     float a, b;
16     Rectangle(float w, float h) : a(w), b(h) {}
17 };
18
19 // Define the variant holding the possible types
```

```

20 using ShapeVariant = std::variant<Circle, Rectangle>;
21
22 // The Visitor is a Functor with overloaded call operators
23 struct SaveVisitor {
24     std::string filePath;
25     explicit SaveVisitor(const std::string& path) : filePath(path) {}
26
27     void operator()(const Circle& c) const {
28         std::cout << "Saving Circle (radius " << c.radius << ") to " <<
filePath << "\n";
29     }
30
31     void operator()(const Rectangle& r) const {
32         std::cout << "Saving Rectangle to " << filePath << "\n";
33     }
34 };
35
36 void runModernVisitor() {
37     std::vector<ShapeVariant> shapes;
38     shapes.push_back(Circle{5.0f});
39     shapes.push_back(Rectangle{4.0f, 6.0f});
40
41     SaveVisitor saveOp("modern_data.xml");
42
43     for (const auto& shape : shapes) {
44         // std::visit handles the runtime dispatching
45         std::visit(saveOp, shape);
46     }
47 }

```

Listing 6: Modern Visitor implementation using `std::variant` and `std::visit`.

5.2 The Acyclic Visitor Pattern

5.2.1 Structural Overview

The Acyclic Visitor pattern alters the class hierarchy by utilizing a degenerate (empty) base visitor class. Instead of a single, monolithic `Visitor` interface that declares `visit` methods for all elements, the Acyclic pattern establishes discrete, independent visitor interfaces for each concrete element (e.g., `CircleVisitor`, `RectangleVisitor`). A concrete visitor class then inherits from the degenerate base and selectively implements only the interfaces corresponding to the elements it intends to process.

Figure 5.2: UML Class Diagram of the Acyclic Visitor Pattern.

5.2.2 Mechanism: Cross-Casting

The core dispatch mechanism in the Acyclic Visitor relies on C++ Run-Time Type Information (RTTI) through cross-casting.

When the client invokes `accept(VisitorBase&)` on an element, the element receives the degenerate base reference. Inside the `accept` method, the concrete element attempts to execute a `dynamic_cast` to cast the incoming base visitor reference into its own specific visitor interface pointer (e.g., `Circle` attempts to cast to `CircleVisitor*`). If the cast is

successful (meaning the concrete visitor implemented that specific interface), the element calls the `visit` method. If the cast fails, the operation is safely bypassed.

```
1 #include <iostream>
2
3 // 1. Degenerate Base Visitor
4 class VisitorBase {
5 public:
6     virtual ~VisitorBase() = default;
7 };
8
9 // 2. Specific Visitor Interfaces
10 class Circle;
11 class CircleVisitor {
12 public:
13     virtual void visit(Circle& c) = 0;
14 };
15
16 class Rectangle;
17 class RectangleVisitor {
18 public:
19     virtual void visit(Rectangle& r) = 0;
20 };
21
22 // 3. Element Hierarchy
23 class Shape {
24 public:
25     virtual ~Shape() = default;
26     virtual void accept(VisitorBase& v) = 0;
27 };
28
29 class Circle : public Shape {
30 public:
31     void accept(VisitorBase& v) override {
32         // Cross-cast execution
33         if (auto* cv = dynamic_cast<CircleVisitor*>(&v)) {
34             cv->visit(*this);
35         }
36     }
37 };
38
39 class Rectangle : public Shape {
40 public:
41     void accept(VisitorBase& v) override {
42         // Cross-cast execution
43         if (auto* rv = dynamic_cast<RectangleVisitor*>(&v)) {
44             rv->visit(*this);
45         }
46     }
47 };
48
49 // 4. Concrete Visitor (Selectively implements specific interfaces)
50 class SaveVisitor : public VisitorBase, public CircleVisitor {
51 public:
52     // Only implements visit for Circle. Rectangle will fail the
53     // dynamic_cast silently.
54     void visit(Circle& c) override {
55         std::cout << "Acyclic: Saving Circle\n";
```

56 };

Listing 7: Acyclic Visitor Pattern leveraging cross-casting via `dynamic_cast`.

6 Evaluation

To quantitatively analyze the runtime efficiency of the three discussed Visitor pattern architectures, an empirical benchmark was conducted. This section outlines the testing environment, measurement methodology, and the recorded results.

6.1 Experimental Environment

To ensure transparency and reproducibility, the hardware and software configurations used for the benchmark are specified below:

- **Operating System:** Windows
- **CPU:** Intel(R) Core(TM) Ultra 7 255H
- **Compiler:** GCC `g++.exe` (Rev8, Built by MSYS2 project) 15.2.0
- **Optimization Flag:** `-O3` (to enable maximum performance optimizations)

6.2 Measurement Methodology

The benchmark was carefully designed to prevent compiler optimization traps and hardware biases:

- **Dataset:** A pre-allocated array consisting of 10,000,000 `Shape` objects (specifically `Circle` and `Rectangle`).
- **Branch Predictor Neutralization:** The array was strictly randomized using the `std::mt19937` random number generator to defeat the CPU's branch predictor.
- **Execution Strategy:** Each architectural variant was executed independently across 3 separate runs. Within each run, the system iterated through the entire 10-million-element array 20 times. The minimum execution time was extracted to filter out background system noise.

6.3 Disclaimer

Disclaimer: The performance metrics presented in this evaluation are strictly for reference purposes. They represent the behavior of the architectures under a specific micro-benchmark scenario. These results should not be interpreted as absolute assertions, as execution times can fluctuate significantly depending on the target hardware, compiler version, memory architecture, and the complexity of the real-world payload. Notably, this benchmark does not completely isolate the dispatch cost from memory-layout effects. In particular, the `std::variant` implementation stores objects by value in contiguous memory, whereas the Classic and Acyclic Visitor implementations use polymorphic pointers on the heap. Therefore, the measured differences reflect both dispatch mechanisms and data-locality characteristics.

6.4 Benchmark Results

The following table summarizes the execution times across the 3 independent runs, highlighting the minimum (best) time for a single iteration over the 10 million elements:

Table 6.1: Benchmark results for processing 10,000,000 objects.

Architecture	Run 1 (ms)	Run 2 (ms)	Run 3 (ms)	Best Time (ms)
Modern Visitor (<code>std::variant</code>)	44.07	42.81	43.39	42.81
Classic Visitor (Double Dispatch)	292.68	261.37	228.11	228.11
Acyclic Visitor (<code>dynamic_cast</code>)	4134.47	3953.18	4449.97	3953.18

7 Discussion

This section critically evaluates the three discussed architectural approaches for implementing the Visitor pattern. The comparison focuses on runtime performance, structural design, adherence to SOLID principles, and ease of maintenance.

7.1 Classic Visitor Pattern

Advantages:

- **Extensibility of Operations:** It facilitates the seamless addition of new operations (Visitors) without modifying the existing codebase of the element hierarchy, strictly adhering to the Open/Closed Principle (OCP) in terms of behaviors.
- **Cross-Hierarchy Processing:** The `Visitor` interface can declare `visit` methods for concrete elements that do not share the same abstract base class, allowing operations to span multiple disparate class hierarchies.

Disadvantages:

- **Runtime Overhead:** The double-dispatch mechanism necessitates two virtual function calls (vtable lookups) per element. This indirection inherently slows down execution compared to a standard single dispatch.
- **Cache Inefficiency:** Classic elements are typically managed via pointers and allocated dynamically on the heap. This scattered memory layout causes frequent CPU cache misses, further degrading runtime performance.
- **Intrusive Architecture:** Implementing this pattern requires injecting the `accept` method into every element class. This makes it an unviable solution if the developer lacks access to the original source code (e.g., when dealing with third-party libraries).
- **Type-Addition Bottleneck:** The design is fundamentally unsuited for systems where new element types are frequently added. Introducing a new concrete element requires modifying the base `Visitor` interface, which subsequently forces a recompilation of all existing visitor implementations, violating the OCP for types.
- **Circular Dependency and Code Duplication:** The pattern introduces a rigid circular dependency between the element header files and the visitor header files. Furthermore, it often forces developers to write boilerplate code, repeating similar `visit` logic for different elements.

7.2 Modern C++ Approach (`std::variant` and `std::visit`)

Advantages:

- **Observed Runtime Advantage in the Benchmark:** By storing objects by value, `std::variant` ensures contiguous memory allocation (cache-friendly). Furthermore, it utilizes compile-time type-based visitation over a closed set of alternatives rather than traditional vtables, reducing dispatch overhead in common scenarios.
- **Non-Intrusive Design:** The elements remain entirely independent. There is no need to define a base class or implement an `accept` method, allowing this approach to work seamlessly with third-party or legacy classes.
- **Flexible Subsetting:** Developers can define a variant using a specific subset of derived classes rather than being forced to depend on the entire hierarchy.
- **Code Deduplication:** By leveraging generic lambdas (`auto`) or templated `operator()` methods inside the visitor functor, developers can handle multiple types implicitly, significantly reducing repetitive code (adhering to the DRY principle).

Disadvantages:

- **Violation of the Dependency Inversion Principle (DIP):** Unlike traditional OOP where high-level modules depend on abstractions (e.g., `Shape*`), `std::variant` requires hardcoding the exact list of concrete types (e.g., `std::variant<Circle, Rectangle>`). This couples the design directly to concrete implementations.
- **Type-Addition Bottleneck (OCP Violation):** Similar to the Classic Visitor, adding a new type requires modifying the `std::variant` type alias and updating the functor to handle the new type, which triggers extensive recompilation.

7.3 The Acyclic Visitor Pattern

Advantages:

- **Decoupled Architecture:** It entirely eliminates the circular dependency problem found in the Classic Visitor by utilizing a degenerate base interface.
- **Safe and Independent Evolution:** Acyclic Visitor reduces the coupling between the central visitor interface and individual element types, allowing new element-specific visitor interfaces to be introduced without modifying a monolithic Visitor interface. This prevents cascading compilation errors when extending parts of the system.
- **Selective Implementation:** Concrete visitors are only required to inherit and implement the specific visitor interfaces for the elements they actually intend to process, avoiding bloated “catch-all” interfaces.

Disadvantages:

- **Severe Runtime Overhead:** The core dispatch mechanism relies on cross-casting via `dynamic_cast`. Run-Time Type Information (RTTI) lookups are notoriously slow in C++, making this pattern unsuitable for performance-critical applications (e.g., game engines or high-frequency trading).

- **Retained Intrusiveness:** Despite fixing the dependency cycle, the Acyclic Visitor still requires element classes to implement the `accept` method, maintaining an intrusive footprint on the data structures.

8 Real-World Applications

The Visitor pattern excels in domains where complex, stable object structures require frequent application of new, distinct operations. Its ability to decouple algorithms from data structures makes it a foundational architectural choice in several critical software engineering fields.

8.1 Compiler Construction and AST Processing

One of the most canonical applications of the Visitor pattern is in compiler design, transpilers, and code analysis tools. During processing, the source code is parsed into an **Abstract Syntax Tree (AST)**—a deeply nested, heterogeneous tree where each node represents a syntactic construct (e.g., `IfStatement`, `VariableDeclaration`).

Because the grammar of a programming language is highly stable, the AST structure rarely changes. However, the operations performed on the AST are numerous and frequently updated: type-checking, code minification, linting, and code generation.

By employing the Visitor pattern, these tools can define operations like a `TypeCheckVisitor` or `CodeGenerationVisitor` that traverse the AST and execute specific logic for each node type, completely avoiding the pollution of the AST node classes with operational logic.

8.2 GUI Hierarchies and Theming Engines

In some application development contexts, graphical user interfaces (GUIs) are constructed using the Composite pattern, forming a tree of diverse UI elements (e.g., `Button`, `TextView`, `Panel`).

A core principle of clean architecture is that UI components should strictly encapsulate their state and intrinsic behavior, without being burdened by external concerns. For instance, injecting logic to apply a "Dark Mode" directly into a `Button` class can violate the Single Responsibility Principle.

While not universally adopted by all modern GUI frameworks, some architectures employ the Visitor pattern to address this. A `ThemeVisitor` or `RenderVisitor` can traverse the UI component tree. As it visits each node, it applies styling properties based on the specific type of the component, ensuring that the visual logic remains decoupled from the structural components.

8.3 Document Serialization and Exporting

Enterprise software frequently deals with complex domain models, such as CAD drawings, vector graphics (as demonstrated in the `Shape` examples earlier), or rich text documents. A persistent requirement in such systems is the ability to export these object structures into various formats, such as XML, JSON, or PDF.

Rather than forcing every domain object to implement `exportToXML()` and `exportToJSON()`, a `SerializationVisitor` is created for each target format. This approach allows developers to introduce new export formats seamlessly by simply adding a new concrete visitor, fully satisfying the Open/Closed Principle while keeping the core domain models lean and focused solely on business logic.

9 Quiz

To reinforce the concepts discussed during the seminar, the following multiple-choice questions have been designed. They cover the fundamental mechanics of the Classic Visitor pattern, as well as the architectural traits of its modern variants.

1. **What type of dispatch is naturally applied by C++, and what type of dispatch is simulated by the Visitor pattern?**

- A. Double-dispatch, single-dispatch
- B. Both single-dispatch
- C. Both double-dispatch
- D. Single-dispatch, double-dispatch

Answer: D

2. **When is the BEST time to use the Visitor pattern?**

- A. When the classes defining the object structure change frequently.
- B. When the object structure is completely stable, but you often need to define new operations over it.
- C. When you need to hide the creation logic of an object.
- D. When you want to pass a request along a chain of objects until it is handled.

Answer: B

3. **What is one of the main negative consequences (drawbacks) of using the Visitor pattern?**

- A. It makes adding new operations incredibly difficult.
- B. It can break encapsulation by forcing elements to expose their internal state via public methods.
- C. It prevents visitors from accumulating state during traversal.
- D. It forces all elements to share a single, strict parent class.

Answer: B

4. **How does the modern `std::variant` approach eliminate the need for an abstract base class and the intrusive `accept` method?**

- A. By using `dynamic_cast` to resolve types at runtime recursively.

- B. By relying on compile-time type-based visitation over a closed set of alternatives and invoking an overloaded `operator()` inside a Functor.
- C. By implementing the double-dispatch mechanism exclusively through `virtual` functions.
- D. By forcing all elements to inherit from a common `std::variant` interface.

Answer: B

5. **What is the primary architectural technique utilized by the Acyclic Visitor pattern to break the circular dependency between elements and visitors?**
- A. Utilizing a single degenerate (empty) base interface combined with runtime cross-casting (`dynamic_cast`).
 - B. Extracting all `visit` methods into a standalone `std::visit` function.
 - C. Using multiple inheritance for the Element classes instead of the Visitor classes.
 - D. Storing elements contiguously in memory using `std::vector` to avoid pointer dependencies.

Answer: A

6. **According to the benchmark, what primarily contributes to the observed runtime advantage of the `std::variant` Visitor over the Classic and Acyclic Visitors?**
- A. It utilizes RTTI lookups (`typeid`), which are heavily optimized by modern CPUs.
 - B. It executes fewer instructions by avoiding `dynamic_cast` while continuing to use heap-allocated pointers.
 - C. It runs in a multi-threaded context automatically via standard C++17 library optimizations.
 - D. It stores objects by value, maximizing cache locality (cache hits), and entirely avoids virtual table (vtable) indirection.

Answer: D

7. **If new element types are expected to be added frequently, which approach is generally more suitable?**
- A. Classic Visitor, because it strictly adheres to the Open/Closed Principle for both operations and types.
 - B. `std::variant`, because it easily supports adding new types without modifying the variant alias or recompiling.
 - C. Acyclic Visitor, because it allows introducing new element-specific visitor interfaces without modifying a monolithic Visitor interface.
 - D. None of them, as the Visitor pattern is fundamentally flawed when dealing with new operations.

Answer: C

10 Conclusion

In conclusion, the Visitor design pattern provides a robust, OOP-compliant architectural solution for scenarios where an object hierarchy is highly stable, yet the operations performed upon it are subject to frequent expansion. By elegantly decoupling algorithms from the underlying data structures, the pattern effectively upholds the Open/Closed Principle in terms of operational extensibility.

However, as analyzed throughout this report, the Classic Visitor pattern and its advanced variants are not universal panaceas. Each architectural approach introduces distinct engineering trade-offs that must be meticulously evaluated prior to implementation:

- The **Classic Visitor** provides standard structural decoupling and strict type safety but suffers from rigid circular dependencies and the moderate overhead of double virtual dispatching.
- The **Modern `std::variant` Visitor** (C++17) delivers unparalleled runtime performance and optimal cache locality by completely eliminating virtual method tables. It is highly recommended for modern, performance-critical C++ codebases, despite its compromise on the Dependency Inversion Principle.
- The **Acyclic Visitor** successfully resolves the dependency cycle issue, making it suitable for massive, highly decoupled legacy systems. Nevertheless, the severe runtime penalties associated with `dynamic_cast` restrict its viability in latency-sensitive applications.

Ultimately, selecting the appropriate Visitor variant—or deciding whether to employ the pattern at all—requires software architects to rigorously assess the specific constraints of their domain. If the core element hierarchy is volatile and prone to frequent additions, the Visitor pattern will inevitably induce architectural fragility and should be avoided. Conversely, for stable data structures demanding continuous operational growth (such as ASTs or Document Object Models), it remains an indispensable and highly effective tool within the modern software engineering arsenal.

11 References

- [1] Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional, 1994.
- [2] Klaus Iglberger. *C++ Software Design: Design Principles and Patterns for High-Quality Software*. O'Reilly Media, 2022.
- [3] cppreference.com contributors. `std::variant` - cppreference.com. <https://en.cppreference.com/w/cpp/utility/variant>, 2024.
- [4] cppreference.com contributors. `std::visit` - cppreference.com. <https://en.cppreference.com/w/cpp/utility/variant/visit>, 2024.
- [5] cppreference.com contributors. `dynamic_cast` conversion - cppreference.com. https://en.cppreference.com/w/cpp/language/dynamic_cast, 2024.
- [6] Robert C. Martin. Acyclic visitor. *Pattern Languages of Program Design 3*, 1997.

A Declaration of AI Assistance

The authors acknowledge the use of Large Language Models (LLMs) to assist in the preparation of this report. Specifically, AI tools were utilized for the following purposes:

- Assisting with $\text{\LaTeX}$ formatting and document structuring.
- Refining the academic tone and grammatical structure of the English text.
- Aiding in the synthesis of structural explanations for the discussed C++ architectural patterns.

We confirm that the AI was strictly used as an assistive writing and formatting tool. All core concepts, architectural diagrams, benchmark designs, and code implementations were developed, thoroughly reviewed, and validated by the human authors. The authors assume full responsibility for the accuracy, originality, and technical correctness of the final content presented in this document.